Manual: run_initial_doe.py

`run_initial_doe.py` is the Spyder-friendly launcher for DOE and optimization studies. It can generate initial DOE candidates, turn them into DEFORM runs, process completed results, train a surrogate, propose new candidates, and refresh visualization reports.

The same workflow can also be launched from the DOE tab in `deform_config_gui.py`, which writes a temporary GUI launcher and applies GUI-selected overrides.

See the standalone flowchart: run_initial_doe_flowchart.md.

Main idea

The DOE pipeline separates planning from execution. First, it creates candidate rows and generated inputs. Then, only when `EXECUTE_RUNS` is true, it launches the DEFORM runner. After DEFORM finishes, the processing step builds a dataset from completed runs only. Bayesian proposal uses that dataset to suggest the next finite batch of runs.

User settings

Edit the settings at the top of `run_initial_doe.py` before running it in Spyder, or use the GUI DOE tab to override them.

- `PROJECT_ROOT`: folder containing the DEFORM automation code.
- `RUNNER_SCRIPT`: path to `run_deform_hits_config.py`.
- `BASE_CONFIG_PATH`: the normal runner config used as the starting point.
- `OPTIMIZATION_ROOT`: parent folder for DOE candidates, generated inputs, runs, manifests, datasets, and reports.
- `NUM_HITS`: number of hits per generated run.
- `STUDY_NAME`: name for the initial DOE study folder.
- `BOUNDS`: design variables and lower/upper bounds.
- `OBJECTIVE_WEIGHTS`: weights used to combine processed metrics into a scalar objective.

Control flags

- `PREPARE_RUNS`: generate DOE candidates, action CSVs, config JSONs, and the manifest.
- `EXECUTE_RUNS`: run DEFORM for each generated config.
- `PROCESS_RESULTS`: process completed run folders and build `dataset.csv`.
- `PROPOSE_NEXT_CANDIDATES`: train the surrogate and write `candidates_next.csv`.
- `RUN_NEXT_CANDIDATES`: create action/config files for proposed candidates.
- `EXECUTE_NEXT_CANDIDATES`: run the proposed-candidate batch.
- `PROCESS_NEXT_CANDIDATES`: process completed proposed-candidate runs.
- `COMBINE_DATASETS`: combine available round datasets into `dataset_all.csv`.
- `MAKE_REPORT`: refresh the HTML report and plots.

The current launcher runs a finite batch, not an infinite loop. The guard `NEXT_ROUND_NUMBER` and `MAX_ROUND_NUMBER` prevents accidental open-ended runs.

Recommended first-run workflow

1. Set `PREPARE_RUNS = True` and `EXECUTE_RUNS = False`.
2. Run the file in Spyder or use the GUI DOE tab.
3. Inspect generated action CSVs and config JSONs under `optimization/study_name/inputs`.
4. Set `EXECUTE_RUNS = True` only when the generated files look correct.
5. After DEFORM runs finish, set `PROCESS_RESULTS = True`.
6. Set `PROPOSE_NEXT_CANDIDATES = True` when `dataset.csv` has enough completed rows to train on.
7. Set `RUN_NEXT_CANDIDATES = True` to generate the proposed batch.
8. Set `EXECUTE_NEXT_CANDIDATES = True` when ready to run that batch.
9. Set `MAKE_REPORT = True` whenever the report should be refreshed.

DOE generation

Initial candidates are generated by `optimizers/doe.py` using Latin-hypercube sampling. The resulting `candidates_initial.csv` contains one candidate per planned DEFORM run.

`doe_runner.py` converts candidates into concrete run inputs. For each candidate it writes:

- one candidate-specific action CSV;
- one candidate-specific runner config JSON;
- one manifest row with planned run metadata.

Candidate columns can modify either action CSV columns or config keys. The default `APPLY_MODE = "all_rows"` applies candidate values to every hit row in the action CSV. `APPLY_MODE = "hit_row"` can be used when the candidate file includes a `hit` column.

Result processing

`process_doe_results.py` reads the manifest and processes rows with status `completed` by default. It extracts or records:

- initial microstructure from the starting key file;
- final/post-hit microstructure CSVs;
- damage summaries when DEFORM post-processing access is available;
- coordinate CSV paths;
- load-stroke CSV paths;
- objective terms and scalar objective values.

The processed output becomes `dataset.csv`. Planned, running, and failed rows are intentionally ignored unless the status filter is changed.

Bayesian proposal

When `PROPOSE_NEXT_CANDIDATES` is true, `train_and_propose_next` trains the configured surrogate model and writes `candidates_next.csv`. If `dataset_all.csv` exists, it is used for training; otherwise the initial `dataset.csv` is used.

The proposal step needs enough completed rows and the required Python packages for the selected model type.

Visualization

When `MAKE_REPORT` is true, `visualize_doe_results.py` writes an HTML report and plot images under the study report folder. The GUI Post/Reports tab can run the visualization script and display generated PNG plots inside the GUI.

Referenced code files

- `deform_config_gui.py`: GUI wrapper for DOE launch and report viewing.
- `doe_runner.py`: turns candidates into action/config files and optionally executes runs.
- `process_doe_results.py`: converts completed run outputs into a dataset.
- `dataset_builder.py`: builds model-ready datasets from manifest metadata and processed metrics.
- `surrogate_model.py`: trains surrogate models and writes training reports.
- `optimizers/doe.py`: initial candidate generation.
- `optimizers/bayesian.py`: Bayesian candidate proposal.
- `visualize_doe_results.py`: plots and HTML reporting.